

Model Credit Framework

A practical, role-based guide for assigning models, controlling token/AI credit usage, reducing duplicate tool spend, and guiding developers on when to use lightweight, standard, and premium AI models.

Model Access

AI Credits

Role-Based Guidance

Daily Usage Rules

Premium Model Control

Table of Contents

1. Objective	2. Background
3. Key Problem Statement	4. Guiding Principle
5. GitHub Copilot Model Catalogue	6. Model Categories
7. Role-Based Access Matrix	8. Recommended Access Tiers
9. Usage Cost, Tokens & AI Credits	10. Suggested User-Level Budget Approach
11. Daily Model Usage Guidelines	12. Normal vs Premium Examples
13. AWS Kiro & Copilot Rationalization	13A. Kiro vs GitHub Copilot - Simple Comparison
14. Approval Process for Premium Models	15. Monitoring & Review Cadence
16. Efficient Usage Rules	17. Default Policy & Implementation Plan
18. Final Recommendation & Leadership Decision	19. Developer & Manager Communication
20. References	

3

Model groups

Lightweight, Standard, Premium

5

Access tiers

Tier 0 through Tier 4

1 rule

Default guidance

Start normal, escalate only when needed

1. Objective

The objective of this document is to define a clear governance approach for GitHub Copilot model access, AI credit usage, premium model allocation, and license rationalization across engineering teams.

The goal is to ensure that the right users receive the right AI models based on their role, project complexity, delivery responsibility, and actual usage pattern, while avoiding unnecessary cost leakage caused by overlapping tools such as GitHub Copilot and AWS Kiro.

This framework helps the organization

- Control AI spend within the approved monthly budget.
- Avoid unnecessary usage of high-cost premium models.
- Prevent individual users from exhausting shared AI credit pools too early.
- Ensure normal developers get sufficient models for day-to-day productivity.

It also helps with governance

- Provide premium model access only to users with genuine business or technical need.
- Reallocate Copilot licenses where AWS Kiro is already actively used.
- Improve transparency for managers, engineering leads, and finance stakeholders.

2. Background

The current monthly budget is acceptable from a finance perspective. However, the concern is not only the total budget amount, but how effectively the budget is being consumed across users, models, and teams.

GitHub Copilot provides multiple AI models with different capability and cost profiles. Some models are lightweight and suitable for routine development tasks, while others are premium or frontier models intended for complex reasoning, architecture, debugging, refactoring, and agentic coding workflows.

At the same time, several developers also have access to AWS Kiro. If a developer is actively using Kiro and also has Copilot premium model access, there may be duplication of tooling, overlapping spend, and reduced clarity on which tool is actually driving productivity.

The structured access model should define

- Who should get Copilot access.
- Who should get only standard models.
- Who should get premium models.
- Who should be restricted to lightweight models.
- Who should be moved to Kiro-only access.
- How AI credits or token usage should be controlled.
- How access should be reviewed and reallocated periodically.

3. Key Problem Statement

The main issue is not only license cost. The bigger issue is uncontrolled AI model consumption.

A standard developer may only need a normal coding model for daily development, unit test generation, code explanation, and small bug fixes. However, if all developers are allowed to use premium models freely, they may consume a large portion of the shared AI credit pool quickly.

1. Budget Exhaustion

Premium models consume more credits or requests depending on the billing model. If used casually, the shared allocation can be exhausted early in the month.

2. Productivity Escalation Risk

Once limits are exhausted, developers or managers may raise delivery-risk concerns, even if the usage was not aligned to project priority.

3. Tool Overlap

Developers with both AWS Kiro and GitHub Copilot may use both in parallel without a defined need, creating duplicate spend.

4. Lack of Accountability

Without a model access policy, there is no clear way to justify why one user has premium model access and another does not.

4. Guiding Principle

AI model access should be role-based, need-based, and usage-reviewed.

Default approach

- Standard developers should use standard or lightweight models.
- Premium models should be reserved for senior engineers, tech leads, architects, complex debugging, production issues, large refactoring, and critical delivery needs.
- Developers actively using AWS Kiro should not automatically retain full GitHub Copilot access unless there is a justified project need.

Governance approach

- User-level budgets should be configured to avoid one heavy user consuming an unfair share of the common pool.
- Model access should be reviewed monthly based on actual usage.
- Start with a cheaper or standard model and escalate only if the result is insufficient.

5. GitHub Copilot Model Catalogue

The currently documented Copilot model list includes lightweight, versatile, and powerful models across OpenAI, Anthropic, Google, Microsoft, and GitHub fine-tuned models. Availability can vary by plan, client, and organizational model policy.

Model	Provider	Status	Cost / Capability Group	Recommended Usage Guidance
GPT-5 mini	OpenAI	GA	Lightweight	Simple coding help, quick explanations, low-cost daily prompts
GPT-5.3-Codex	OpenAI	GA	Powerful coding/base/LTS	Coding-heavy tasks, repository understanding, larger changes when justified
GPT-5.4	OpenAI	GA	Versatile	General development, reasoning, refactoring; use long context only when needed
GPT-5.4 mini	OpenAI	GA	Lightweight	Daily developer use, small/medium code changes
GPT-5.4 nano	OpenAI	GA	Lightweight	Very small/simple tasks where available
GPT-5.5	OpenAI	GA	Powerful	Deep reasoning, architecture, complex debugging, high-risk work
Claude Haiku 4.5	Anthropic	GA	Versatile	Fast explanations, small tasks, learning and documentation
Claude Sonnet 4.5	Anthropic	GA	Versatile	Standard development model for normal coding work
Claude Sonnet 4.6	Anthropic	GA	Versatile	Default standard model for most developers; replaces older Sonnet references
Claude Opus 4.5	Anthropic	GA	Powerful	Premium reasoning, complex debugging, architecture
Claude Opus 4.6	Anthropic	GA	Powerful	Premium high-risk work and multi-file reasoning
Claude Opus 4.6 fast mode	Anthropic	Public preview	Powerful	Faster premium work where preview use is acceptable
Claude Opus 4.7	Anthropic	GA	Powerful	Premium deep reasoning and critical tasks
Claude Opus 4.8	Anthropic	GA	Powerful	Premium deep reasoning and critical tasks
Claude Fable 5	Anthropic	GA / currently unavailable note	Powerful	Do not enable broadly; review data handling and availability

Model	Provider	Status	Cost / Capability Group	Recommended Usage Guidance
Gemini 2.5 Pro	Google	GA	Powerful	Complex reasoning and analysis where justified
Gemini 3 Flash	Google	Public preview	Lightweight	Fast daily help and low-risk tasks
Gemini 3.1 Pro	Google	Public preview	Powerful	Premium/pro tasks, large analysis, deep reasoning
Gemini 3.5 Flash	Google	GA	Lightweight	Fast standard work, simple tasks
MAI-Code-1-Flash	Microsoft	GA	Lightweight	Fast coding help and daily prompts
Raptor mini	Fine-tuned GPT-5 mini	Public preview	Versatile	Quick coding tasks and auto-selection use cases

Important: If any internal reference says "Claude Sonnet 3.6" or older Sonnet versions, replace it with the currently supported Sonnet model available in the organization, such as Claude Sonnet 4.6 or Claude Sonnet 4.5.

6. GitHub Copilot Model Categories

For governance purposes, models should be grouped into three broad categories.

Category A: Lightweight / Low-Cost Models

Default or safe models for routine work.

- Small code suggestions
- Code explanation
- Unit test skeletons
- Simple bug fixes
- Documentation
- Small script generation
- Basic code cleanup
- Learning and onboarding support

Example models: GPT-5 mini, GPT-5.4 nano, GPT-5.4 mini, Gemini 3 Flash, MAI-Code-1-Flash, Raptor mini, Claude Haiku 4.5.

Access recommendation:
Enable for most Copilot users.

Category B: Standard / General Development Models

Default choice for most active developers.

- Day-to-day coding
- Medium-complexity debugging
- API development
- Unit test generation
- Code refactoring within a module
- Pull request explanation
- Developer productivity support
- Moderate reasoning tasks

Example models: Claude Sonnet 4.6, Claude Sonnet 4.5, GPT-5.4, Gemini 3.5 Flash, Gemini 2.5 Pro where justified, GPT-5.3-Codex where approved.

Access recommendation:
Assign to normal developers, senior developers, QA automation, DevOps, and active delivery teams.

Category C: Premium / High-Cost / Frontier Models

Restricted and controlled models for high-impact work.

- Complex architecture decisions
- Large multi-file refactoring
- Production incident debugging
- Deep reasoning
- Performance optimization
- Security remediation planning
- Critical release support
- Complex migration work
- Cross-system design analysis
- Agentic workflows with high business impact

Example models: Claude Opus 4.5/4.6/4.7/4.8, Claude Opus 4.6 fast mode, GPT-5.5, Gemini 3.1 Pro, Claude Fable 5 where available and approved.

Access recommendation: Do not enable for all developers by default.

7. Role-Based Access Matrix

Role / User Group	Recommended Access	Premium Model Access	Suggested Budget Control	Remarks
Interns / Trainees	Lightweight models only	No	Low	Use for learning, code explanation, and small tasks only
Junior Developers	Lightweight + standard model	No by default	Low to medium	Claude Sonnet or equivalent standard model is sufficient
Standard Developers	Standard models	No by default	Medium	Suitable for daily coding, testing, and debugging
Senior Developers	Standard models + limited premium access	On approval	Medium to high	Premium access for complex bugs,

Role / User Group	Recommended Access	Premium Model Access	Suggested Budget Control	Remarks
				refactoring, and delivery-critical tasks
Tech Leads	Standard + premium models	Yes, controlled	High	Needed for design reviews, complex debugging, and team-level delivery support
Architects	Premium models	Yes	High	Required for architecture, system design, and complex reasoning
DevOps / SRE	Standard models + limited premium access	On approval	Medium to high	Premium access for incident analysis, automation, CI/CD issues, infra debugging
QA Automation Engineers	Lightweight + standard models	No by default	Low to medium	Standard model enough for test cases, automation scripts, and test data
Security / DevSecOps	Standard + premium access	Yes, controlled	High	Needed for vulnerability analysis and remediation planning
Engineering Managers	Read-only/reporting access or standard model	No by default	Low	Should not consume premium models unless technically hands-on
Product Managers / Scrum Masters	No Copilot or lightweight only	No	Very low	Access only if they need technical documentation support
Developers using AWS Kiro actively	Kiro-only or reduced Copilot access	No by default	Very low or remove Copilot	Avoid duplicate tool spend unless project need is approved

8. Recommended Access Tiers

Tier 0: No Copilot / Kiro-Only

Applicable for:

- Developers actively using AWS Kiro as their primary AI coding tool.
- Non-coding users.
- Inactive Copilot users.
- Users with low or no Copilot usage for the last review period.

Access rule: Remove or pause Copilot license and reallocate to another developer who has active need.

Tier 1: Basic Developer Access

Applicable for:

- Junior developers
- New joiners
- QA engineers
- Developers doing routine maintenance work

Allowed models: Lightweight models and one standard coding model, preferably Claude Sonnet or equivalent.

Not allowed: Claude Opus models, GPT-5.5, high-cost long-context models, premium agentic workflows unless approved.

- Code explanation
- Small bug fixes
- Unit tests
- Small documentation
- Simple refactoring

Tier 2: Standard Engineering Access

Applicable for:

- Regular developers
- Module owners
- Backend/frontend developers
- QA automation engineers
- DevOps engineers with moderate usage

Allowed models: Lightweight models, standard models, and limited access to selected powerful models only when justified.

Premium access: Temporary approval required.

- Feature development

Tier 3: Advanced Engineering Access

Applicable for:

- Senior developers
- Tech leads
- SRE leads
- Security engineers
- Critical project engineers

Allowed models: Lightweight models, standard models, controlled premium models.

Premium access: Allowed with monthly monitoring.

- Complex production debugging
- Large codebase analysis

• Medium debugging • Test generation • API development • Pull request review support • Module-level refactoring	• Critical refactoring • Performance tuning • Security fixes • Complex CI/CD issues • Cross-module impact analysis
---	--

Tier 4: Premium / Strategic Access

Applicable for: Architects, principal engineers, platform leads, engineering leads working on critical programs, and approved innovation or modernization teams.

Allowed models: Full model catalogue subject to policy, premium models, long-context and advanced reasoning models.

Premium access: Allowed, but usage must be tracked and justified.

- Architecture planning
- Large-scale modernization
- Migration strategy
- Complex design trade-off analysis
- Multi-system debugging
- AI-assisted engineering accelerators
- Critical release support

9. Usage Cost, Tokens & AI Credits

1 credit = \$0.01 USD GitHub AI Credits convert token usage into spend.	1,900 Business credits/user/month Included credits are pooled at billing entity level.	3,900 Enterprise credits/user/month Enterprise has a higher monthly allowance.
--	---	---

How cost works in simple terms

- Each Copilot interaction consumes input tokens, output tokens, and sometimes cached tokens.
- Each model has its own per-token price, usually shown per 1 million tokens.
- The total token cost is converted into GitHub AI Credits.
- The cost of an interaction depends on both the selected model and how many tokens are consumed.
- Code completions and next-edit suggestions are not billed in AI credits for paid Copilot plans.
- Long context and higher reasoning can increase token usage, so use them only for complex tasks.

Model	Category	Input / 1M tokens	Cached input / 1M	Cache write / 1M	Output / 1M	Simple Guidance
GPT-5.4 nano	Lightweight	\$0.20	\$0.02	-	\$1.25	Simple/small tasks
GPT-5 mini	Lightweight	\$0.25	\$0.025	-	\$2.00	Daily basic use
Raptor mini	Versatile	\$0.25	\$0.025	-	\$2.00	Quick coding tasks
Gemini 3 Flash	Lightweight	\$0.50	\$0.05	-	\$3.00	Fast daily use
GPT-5.4 mini	Lightweight	\$0.75	\$0.075	-	\$4.50	Daily developer use
MAI-Code-1-Flash	Lightweight	\$0.75	\$0.075	-	\$4.50	Daily developer use
Claude Haiku 4.5	Versatile	\$1.00	\$0.10	\$1.25	\$5.00	Low/medium risk tasks
Gemini 2.5 Pro	Powerful	\$1.25	\$0.125	-	\$10.00	Complex analysis
GPT-5.3-Codex	Powerful	\$1.75	\$0.175	-	\$14.00	Coding-heavy work
Gemini 3.1 Pro	Powerful	\$2.00	\$0.20	-	\$12.00	Premium reasoning
GPT-5.4	Versatile	\$2.50	\$0.25	-	\$15.00	Standard/advanced
Claude Sonnet 4.6	Versatile	\$3.00	\$0.30	\$3.75	\$15.00	Default standard model
	Powerful	\$5.00	\$0.50	\$6.25	\$25.00	Premium only

Model	Category	Input / 1M tokens	Cached input / 1M	Cache write / 1M	Output / 1M	Simple Guidance
Claude Opus 4.6/4.7/4.8						
GPT-5.5	Powerful	\$5.00	\$0.50	-	\$30.00	Premium only
Claude Fable 5	Powerful	\$10.00	\$1.00	\$12.50	\$50.00	Restricted / review

Cost principle: normal daily work should use lightweight or standard models. Premium models should be used for thinking-heavy, high-risk, or critical tasks because they generally have higher token cost and can consume credits faster.

10. Suggested User-Level Budget Approach

The organization should configure user-level budgets to prevent uncontrolled consumption. These numbers should be finalized based on actual usage data, project priority, and the organization's monthly budget. The first month should be treated as a baseline monitoring period.

User Category	Suggested Monthly AI Credit Budget Equivalent	Purpose
Kiro-active or inactive users	\$0-\$5	Block or minimize duplicate Copilot usage
Interns / trainees	\$5-\$10	Learning and lightweight support
Junior developers	\$10-\$15	Basic coding support
Standard developers	\$15-\$25	Normal delivery support
Senior developers	\$25-\$50	Complex coding and debugging
Tech leads / DevOps leads	\$50-\$75	Team-level support and critical debugging
Architects / Principal engineers	\$75-\$100+	Architecture and strategic work

11. Daily Model Usage Guidelines

Use normal models for daily work. Use premium models only for complex, high-risk, or deep reasoning work.

All developers should use normal or standard models for daily development work. Premium models should not be used for every task because they consume more tokens or AI credits and can increase overall cost quickly.

Daily Development Usage

For regular day-to-day development, developers should use normal models such as Claude Sonnet or other standard coding models available in GitHub Copilot. These models are sufficient for most common development activities.

- Writing small or medium code changes
- Understanding existing code
- Creating unit tests
- Fixing simple bugs
- Explaining code logic
- Writing API methods
- Creating basic documentation
- Refactoring small modules
- Reviewing small pull requests
- Generating simple scripts

For normal developers, standard models like Claude Sonnet should be enough for most daily tasks. Premium models are not required for routine coding.

When to Use Lightweight or Simple Models

Developers should use lightweight or simple models for very basic tasks.

- Syntax help
- Small code snippets
- Code formatting
- Writing comments
- Small documentation updates
- Basic test case generation
- Simple explanation of code
- Learning a new function or library
- Simple debugging

This reduces unnecessary token consumption and keeps premium capacity available for critical users.

When to Use Standard Models

Standard models should be the default choice for most developers.

- Normal feature development
- Medium-complexity debugging

When to Use Premium Models

Premium models should be used only when the task requires deeper reasoning, higher accuracy, or complex code understanding.

- High-risk production issues

- Unit test creation
- API implementation
- Code explanation
- Module-level refactoring
- Pull request review support
- Understanding business logic
- Writing reusable functions
- Improving existing code quality

For most developers, this level of model access is enough.

- Critical bugs blocking delivery
- Complex debugging across multiple files
- Architecture-level decisions
- Large refactoring
- Performance optimization
- Security vulnerability analysis
- Legacy system modernization
- Complex migration work
- Release-critical issues
- Deep reasoning before major design decisions
- Complex root cause analysis

Premium models should be treated as a limited resource and should not be used casually.

12. Examples of Normal vs Premium Usage

Task	Recommended Model Type
Write a small function	Lightweight / Standard
Explain one file	Lightweight / Standard
Create unit tests	Standard
Fix a simple bug	Standard
Refactor one small class	Standard
Review a small pull request	Standard
Debug production issue	Premium
Analyze multiple services	Premium
Plan system architecture	Premium
Optimize performance issue	Premium
Security remediation plan	Premium
Large code migration	Premium

13. AWS Kiro and Copilot Rationalization

Because some developers have access to AWS Kiro, the organization should avoid giving full Copilot access to the same users unless required.

Scenario	Recommended Action
Developer actively uses Kiro for coding and specs	Remove Copilot or restrict to basic access
Developer uses Kiro only occasionally	Keep Copilot standard access, no premium models
Developer works on AWS-heavy project and uses Kiro for implementation	Prefer Kiro access, restrict Copilot premium models
Developer needs Copilot due to IDE/repo workflow	Keep Copilot, but review Kiro usage
Developer is senior/lead and needs both tools	Allow both with manager approval and usage review
No usage in either tool for 30 days	Reclaim license

Combined user list required for review

- GitHub Copilot users
- AWS Kiro users
- Users with access to both
- Actual usage level
- Project mapped to each user
- Manager approval status
- Recommendation: retain, downgrade, remove, or review

13A. Kiro vs GitHub Copilot - Simple Comparison

Purpose of this page: Give a simple one-page view of where GitHub Copilot and AWS Kiro fit, how pricing differs, and which tool should be preferred for different teams.

Area	GitHub Copilot	AWS Kiro
Best fit	Best default tool for daily developer productivity inside GitHub/IDE workflows.	Best for spec-driven, agentic engineering where the tool helps create requirements, design, tasks, code, docs, and tests.
Daily coding	Strong for code completion, chat, small fixes, unit tests, PR help, and quick explanations.	Useful, but more suited when a task needs structured planning before code generation.
Project planning	Good for technical discussion and code-level help.	Stronger for converting prompts into specs, design, task breakdown, implementation, documentation, and tests.
Cost model	AI credits based on model used and tokens consumed. Code completions and next edit suggestions are not billed in AI credits for paid plans.	Credit-based subscription plans. Paid tiers allow overages at published per-credit rates.
Recommended use	Use for most developers as the standard day-to-day coding assistant.	Use selectively for teams/projects that benefit from spec-first engineering or AWS-heavy delivery workflows.

GitHub Copilot pricing - simple view

Business: \$19/user/month
 Enterprise: \$39/user/month

- Copilot Business includes 1,900 AI credits per user/month.
- Copilot Enterprise includes 3,900 AI credits per user/month.
- 1 GitHub AI credit = \$0.01 USD.
- Cost depends on selected model and token usage.
- Strong fit as the default coding assistant for broad engineering teams.

AWS Kiro pricing - simple view

Free: 50 credits
 Pro: \$20/month
 Pro+: \$40/month
 Pro Max: \$100/month
 Power: \$200/month

- Paid tiers provide higher usage capacity.
- Additional overage: \$0.04 per extra Kiro credit.
- Enterprise billing is managed through AWS billing and enterprise setup.
- Strong fit for spec-driven and agentic project execution.

Simple recommendation: GitHub Copilot should remain the default tool for most developers because it is tightly integrated with daily coding, GitHub workflows, code completion, chat, and PR support. Kiro should be used selectively where teams need spec-first agentic engineering, structured project planning, AWS-heavy implementation, or documentation/test generation from requirements.

Cost control note: Do not compare credits directly between Copilot and Kiro because each platform measures usage differently. Compare by actual monthly spend, active usage, developer productivity, and project need.

Source note: Pricing and billing details verified from GitHub Docs and Kiro pricing/billing pages on 23 Jun 2026. Pricing may change, so final procurement decisions should confirm vendor pages before purchase.

14. Approval Process for Premium Model Access

Premium model access should not be permanent by default. It should be approved based on need.

Approval request should include

- User name
- Role
- Project name
- Required model
- Reason for premium access
- Expected duration
- Business justification
- Manager approval
- Review date

Example approval reasons

- Critical production issue
- High-priority release
- Complex code migration
- Architecture review
- Security remediation
- Performance issue
- Large refactoring task

Suggested time-bound premium access

- 7 days for incident support
- 15 days for release support
- 30 days for project delivery
- 90 days for architects or principal engineers, subject to review

15. Monitoring and Review Cadence

The Copilot and Kiro access list should be reviewed every month.

Monthly review should include

- Total Copilot spend
- AI credits consumed
- Usage by user
- Usage by model
- Usage by team or cost center
- Premium model consumption
- Kiro overlap users
- Inactive users
- Users hitting budget limits
- Users with high usage but unclear delivery justification

Review objective

The monthly review should identify which users should continue, which should be downgraded, which licenses should be reclaimed, and where budgets should be adjusted for critical users.

Finding	Action
High usage with valid project need	Continue access
High usage without justification	Restrict model access or reduce budget
Low or no usage	Remove or reallocate license
Kiro + Copilot duplicate access	Rationalize access
Premium model used for simple tasks	Educate user and restrict access
User repeatedly exceeds budget	Manager review required
Critical user blocked due to low budget	Increase individual budget with approval

16. Efficient Usage Guidelines for Developers

Developers should follow these rules to reduce unnecessary token or AI credit consumption.

1. Use code completion and next-edit suggestions for small coding help.
2. Use lightweight models for simple questions.
3. Use standard models for normal development.
4. Use premium models only when the task is complex.
5. Do not paste entire repositories unless needed.
6. Attach or reference only the relevant files.
7. Keep prompts specific and short.
8. Ask for diffs or focused changes instead of full file rewrites.
9. Avoid repeated prompts for the same issue.

10. Start with a cheaper model and escalate only if the result is insufficient.
11. Avoid long-context mode unless working across many files.
12. Do not use AI-generated code without review.
13. Do not share secrets, credentials, tokens, customer data, or confidential information.
14. Use premium models for thinking-heavy tasks, not typing-heavy tasks.
15. Close or reset long sessions when context is no longer needed.

17. Recommended Default Policy and Implementation Plan

Recommended default policy

1. Enable lightweight and standard models for most developers.
2. Disable or restrict premium models by default.
3. Provide premium model access only to approved roles.
4. Configure user-level budgets to prevent excessive consumption.
5. Create individual budget overrides for tech leads, architects, and critical users.
6. Review Copilot and Kiro overlap monthly.
7. Remove Copilot access from active Kiro users unless justified.
8. Track usage by user, model, project, and cost center.
9. Reallocate unused licenses every month.
10. Review policy every quarter as model availability and pricing changes.

Implementation approach

The implementation should move from current-state review to access tier definition, technical controls, enablement, and monthly governance review. This keeps the process simple, measurable, and repeatable.

Phase	Phase Name	Owner	Activities	Output
1	Current State Review	Tarun / Engineering Operations	Export Copilot and Kiro user lists, identify overlap, inactive users, premium users, project mapping, and current spend.	Overlap report, license rationalization recommendation, initial tier mapping
2	Define Access Tiers	Rajesh / Sajid / Engineering Leads	Finalize model categories, role tiers, standard model set, premium approval rules, monthly budget thresholds, and exception process.	Approved model access policy, role-to-model matrix, budget recommendation
3	Configure Controls	GitHub Enterprise Admin / Platform Team	Configure model policies, set user-level budgets, set overrides for power users, create cost center reporting, and restrict premium models.	Controlled Copilot setup, budget guardrails, premium access workflow
4	Communication & Enablement	Engineering Management	Share guidance, explain lightweight/standard/premium usage, model impact, prompting examples, and Kiro vs Copilot expectations.	Developer guide, manager note, escalation process
5	Monthly Governance Review	Engineering Operations / Finance / Engineering Leads	Review usage by user/model/team, spend against budget, high/low usage, premium approvals, Kiro overlap, and license reallocations.	Monthly governance report, action list, forecast

18. Final Recommendation and Suggested Leadership Decision

The organization should adopt a controlled, tier-based access model instead of giving all developers unrestricted access to all Copilot models.

For normal developers, standard models such as Claude Sonnet or equivalent are sufficient for most day-to-day coding tasks. Premium models such as Claude Opus, GPT-5.5, and advanced Gemini Pro models should be restricted to senior engineers, tech leads, architects, DevOps/SRE leads, security engineers, and approved project-critical users.

Developers who actively use AWS Kiro should be reviewed carefully. If Kiro is already meeting their coding and agentic development needs, Copilot access should either be removed, downgraded, or limited to basic models. This approach will help the organization control spend, reduce overlap, protect the shared AI credit pool, and ensure premium AI capability is available to the users and projects where it creates the highest business value.

Approve the following approach

- Proceed with the current monthly budget.
- Immediately review the current Copilot user list.
- Identify users who also have AWS Kiro access.
- Define model access by role.
- Restrict premium models to approved users.
- Set user-level budgets.
- Reclaim unused or duplicate licenses.
- Review usage monthly.
- Publish developer guidelines for efficient AI model usage.

Expected outcome

This creates a balanced approach between productivity and cost control. It gives normal developers enough AI support for daily work while protecting premium model usage for deep reasoning, high-risk work, and critical delivery needs.

19. Developer and Manager Communication

19.1 Suggested Message to Developers

Team,

To manage GitHub Copilot usage effectively and avoid unnecessary token or AI credit consumption, please follow the below model usage guidelines.

For daily development activities, use normal or standard models such as Claude Sonnet or other simple coding models available in Copilot. These models are sufficient for most tasks like coding, unit tests, small bug fixes, code explanation, documentation, and normal refactoring.

Premium models should be used only when there is a genuine need for deep reasoning, complex debugging, high-risk production issues, architecture decisions, security analysis, performance optimization, or critical delivery blockers.

Please avoid using premium models for simple coding help, formatting, basic documentation, small test cases, or routine explanations. Start with a normal model first. Move to a premium model only if the normal model is not able to solve the problem properly.

This approach will help us control spend, avoid unnecessary token usage, and ensure premium models remain available for the teams and projects that genuinely need them.

19.2 Suggested Manager Guidance

Managers and leads should guide their teams to use models responsibly

- Normal developers should use standard models for daily work.
- Junior developers should use lightweight or standard models only.
- Senior developers can use premium models when required.
- Tech leads and architects can use premium models for complex design and issue analysis.
- Developers using AWS Kiro actively should be reviewed before continuing full Copilot access.
- Premium model usage should be reviewed monthly.
- Any repeated high usage should be checked against project need.

19.3 Final Usage Recommendation

Balanced access approach

- Daily coding: normal or standard models.
- Simple tasks: lightweight models.
- Complex/high-risk tasks: premium models.
- Kiro-active users: review and rationalize Copilot access.
- Architects/leads: controlled premium access.
- Normal developers: standard model access is sufficient.

This will help control token usage, reduce cost overlap, and ensure AI tools are used efficiently across the team.

20. References

- GitHub Docs - Supported AI models in GitHub Copilot. Used for model names, providers, release status, auto model selection, extended capabilities, and retirement notes.
- GitHub Docs - Models and pricing for GitHub Copilot. Used for token pricing, AI credit conversion, and statement that code completions/next-edit suggestions are not billed in AI credits for paid plans.
- GitHub Docs - About billing for GitHub Copilot in organizations and enterprises. Used for Copilot Business and Copilot Enterprise prices, included AI credits, pooling, and budget control summary.
- Internal content supplied by the user in the uploaded notes and pasted conversation content.